

Bootcamp Python

Réalisé par Alexandre Coirier

Public : B3 Ingénieurs Informatiques
Durée : 4 séances de 3h30 (14h)
Projet fil rouge : Réseau routier procédural
Projet en groupe : 3 étudiants

Sessions :

1. Rappels Python & environnement
 2. Architecture & génération procédurale
 3. Features & visualisation
 4. Préparation & soutenance
- Grille d'évaluation



SUP^e
VINCI



1. Rappels Python & environnement

Objectifs

- Comprendre le déroulé et les objectifs du module.
- Réactiver les fondamentaux Python nécessaires au projet.
- Installer et valider un environnement de développement commun.
- Être capable d'exécuter un script Python et d'utiliser Matplotlib.

Livrables

- Environnement Python fonctionnel avec VS Code et .venv.
- Matplotlib installé et fenêtre de test fonctionnelle.

Durée

- 3h30

Déroulé

- Introduction au module : déroulé des 4 séances, projet RoadNetwork, livrables et oral.
- Installation et vérification de Python.
- Création d'un environnement virtuel local (.venv).
- Installation de Matplotlib.
- Installation et configuration de VS Code.
- Rappels généraux Python : variables, types, conditions, boucles et fonctions.
- Rappels sur les collections : list, tuple, dict et set.
- Rappels sur les classes, les instances, __init__, self et les imports.
- Premier script et première fenêtre Matplotlib pour valider l'environnement.

1.1 Le module

Les 4 séances

01

Rappels Python & environnement

Remise à niveau, installation et première fenêtre Matplotlib.

02

Architecture & génération procédurale

Construction guidée du socle commun RoadNetwork.

03

Features & visualisation

Évolutions du réseau en autonomie, tests et corrections.

04

Préparation & soutenance

Support, démonstration et oral technique.

Organisation

- Projet réalisé en **groupe de 3**, du socle commun jusqu'à la soutenance.
- Chaque séance de 3h30 alterne apports, construction guidée et travail en groupe.
- Le socle est construit **ensemble** en séance 2, puis chaque groupe le fait évoluer.

Livrables et évaluation

- Une archive ZIP du projet Python.
- Le support de présentation (exporté).
- Une soutenance : présentation, démonstration et questions techniques.
- Le détail des critères figure dans la **grille d'évaluation** en fin de document.

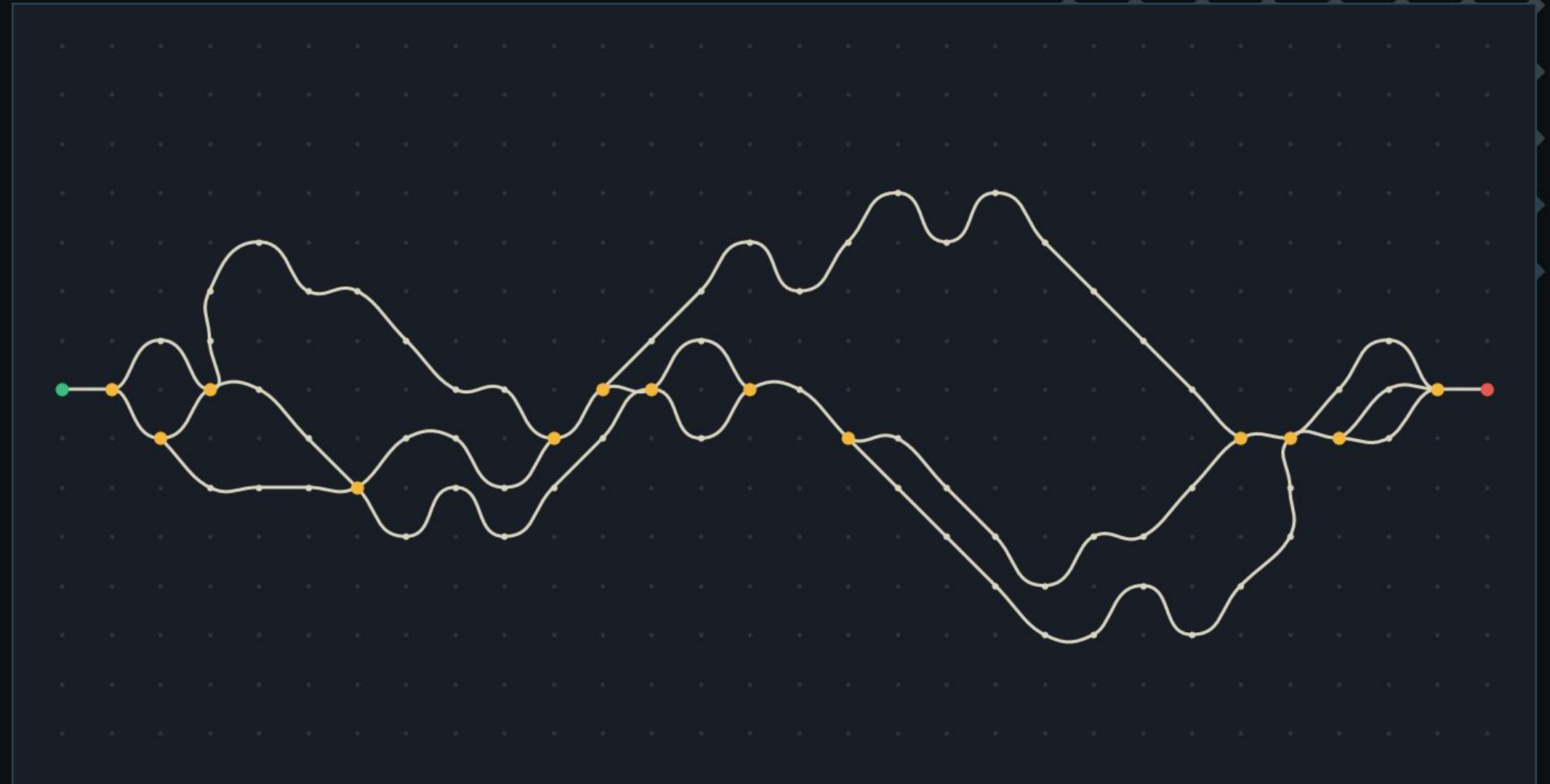
1.1 Le projet RoadNetwork

Le principe

- Générer automatiquement un réseau routier sur une grille, puis l'afficher avec Matplotlib.
- Un réseau routier se représente comme un **graphe** : des **nœuds** reliés par des **segments**.
- Les nœuds sont posés sur une grille de **columns** × **rows** positions.
- Des chemins relient un départ à une arrivée en suivant des règles de génération.

Les types de nœuds

- **Unused** : position libre de la grille.
- **Start** : point de départ du réseau.
- **End** : point d'arrivée du réseau.
- **Connection** : nœud traversé par un chemin.
- **Intersection** : nœud relié à au moins trois segments.



Exemple de résultat attendu en soutenance : plusieurs chemins, intersections et routes courbes.

1.2 Installer Python

Installation selon le système (version du cours : 3.14)

Windows

Installer le `Python install manager` depuis le Microsoft Store ou `python.org`, puis dans un terminal :

```
py install 3.14
```

macOS

Télécharger l'installateur `Python 3.14` sur `python.org` et l'exécuter.

La commande à utiliser est ensuite :

```
python3
```

Linux

Passer par le gestionnaire de paquets. Installer aussi les modules `venv` et `Tk` :

```
sudo apt install python3 \
python3-venv python3-tk
```

Vérifier l'installation

```
> python --version
Python 3.14.3
> python -m pip --version
pip ... from ... (python 3.14)
```

Points d'attention

- Sous macOS et Linux, la commande est souvent `python3` et non `python`.
- Appeler pip via `python -m pip` : on sait quel Python l'exécute.
- Commande introuvable : rouvrir le terminal après l'installation.



1.3 L'environnement virtuel

Pourquoi un .venv ?

- Chaque projet a ses propres bibliothèques et versions, isolées du système.
- Le projet devient **reproductible** : n'importe qui peut réinstaller exactement les mêmes bibliothèques.

Python 3.14 installé sur la machine

`python_b3/.venv`
`matplotlib 3.11.2`

`autre_projet/.venv`
`matplotlib 3.9.4`

L'ordre des opérations

1

Écrire requirements.txt

2

Créer le .venv

3

Installer les dépendances

4

Vérifier

requirements.txt doit exister avant l'étape 3 : c'est ce fichier que pip lit pour savoir quoi installer. Activer le **.venv** est un raccourci facultatif, présenté à la fin du pas à pas.

Les éléments à connaître

.venv/

Le dossier de l'environnement : une copie de Python et les bibliothèques installées. Il se recrée sur chaque machine, il ne se partage pas.

requirements.txt

La liste des bibliothèques du projet. C'est un simple fichier texte, qui se partage avec le code.

.gitignore

Indique à Git les fichiers à ne pas versionner, dont le dossier .venv.

1.3 Créer le .venv pas à pas

Étape

Commandes Windows : dans PowerShell

1

Créer le dossier du projet

Puis ouvrir un terminal à l'intérieur.

```
mkdir python_b3  
cd python_b3
```

2

Écrire requirements.txt

Crée le fichier qui liste les bibliothèques à installer.

```
Windows      Set-Content requirements.txt "matplotlib==3.11.2"  
macOS/Linux echo "matplotlib==3.11.2" > requirements.txt
```

3

Créer l'environnement

Crée le dossier .venv dans le projet.

```
Windows      python -m venv .venv  
macOS/Linux python3 -m venv .venv
```

4

Installer les dépendances

Le Python du .venv lit requirements.txt et installe Matplotlib.

```
Windows      .\.venv\Scripts\python.exe -m pip install -r requirements.txt  
macOS/Linux ./venv/bin/python -m pip install -r requirements.txt
```

5

Vérifier

La commande doit afficher 3.11.2.

```
Windows      .\.venv\Scripts\python.exe -c "import matplotlib; print(matplotlib.__version__)"  
macOS/Linux ./venv/bin/python -c "import matplotlib; print(matplotlib.__version__)"
```

6

Activer (facultatif)

Raccourci : python et pip désignent ensuite ceux du .venv.
L'invite affiche (.venv).

```
Windows      .\.venv\Scripts\Activate.ps1  
macOS/Linux source .venv/bin/activate
```

Sans activation, toujours passer par le Python du **.venv** : un **python** ou un **pip** tapé seul peut viser le Python du système. En cas d'erreur, voir la slide suivante.



1.3 Erreurs fréquentes

Si l'installation bloque

Activate.ps1 s'ouvre dans le Bloc-notes

Cause

Le terminal est l'invite de commandes (cmd), pas PowerShell : la ligne ne commence pas par PS.

```
# Dans cmd, utiliser le script prévu pour cmd
.venv\Scripts\activate.bat
# Ou passer sur PowerShell, dans VS Code :
# Ctrl+Shift+P > Terminal: Select Default Profile > PowerShell
```

L'exécution de scripts est désactivée

Cause

Windows bloque par défaut les scripts .ps1, dont celui d'activation. Le changement s'applique aussitôt, sans rouvrir le terminal.

```
# Autoriser les scripts pour son compte (répondre 0)
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
# Poste verrouillé : autoriser pour ce terminal seulement
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Could not open requirements file

Cause

Le fichier n'est pas dans le dossier courant, ou s'appelle en réalité requirements.txt.txt car Windows masque les extensions.

```
# Vérifier le contenu du dossier et le nom exact du fichier
dir
# Recréer le fichier depuis le terminal si besoin
Set-Content requirements.txt "matplotlib==3.11.2"
```

Pour les deux premiers cas, on peut aussi se passer de l'activation : les commandes directes de la slide précédente fonctionnent sans elle.

1.4 Configurer VS Code

Configuration pas à pas

1 Installer VS Code

Interface

Télécharger VS Code sur code.visualstudio.com et lancer l'installateur.

Windows

Cocher l'option « Open with Code » pour l'ajouter au clic droit.

2 Extension Python

Interface

Cliquer sur l'icône Extensions (quatre carrés) dans la barre de gauche, chercher « Python » et installer celle de Microsoft.

Raccourci

Ctrl+Shift+X

3 Ouvrir le dossier

Interface

Menu File > Open Folder..., puis choisir le dossier du projet. Ou clic droit sur le dossier > Open with Code.

Raccourci

Ctrl+K puis Ctrl+O

4 Choisir l'interpréteur

Interface

Ouvrir un fichier .py, cliquer sur la version de Python affichée en bas à droite et choisir celle du .venv.

Raccourci

Ctrl+Shift+P > Python: Select Interpreter

5 Terminal intégré

Interface

Menu Terminal > New Terminal. Une fois l'interpréteur choisi, le .venv s'y active automatiquement.

Raccourci

Ctrl+ù (AZERTY) ou Ctrl+`

6 Exécuter un script

Interface

Bouton ▶ en haut à droite de l'éditeur, ou menu Run > Run Without Debugging.

Raccourci

Ctrl+F5, ou python main.py dans le terminal

Vérification : la version affichée en bas à droite doit mentionner **.venv**. Les menus sont ceux de VS Code en anglais, langue par défaut. Sous macOS, remplacer Ctrl par Cmd.



1.5 Variables et types

Les types de base

int

Nombre entier : compteurs, index, quantités.

ex : `0`, `20`, `-1`

float

Nombre à virgule : mesures, ratios, positions.

ex : `1.5`, `0.45`, `8 / 3`

str

Chaîne de caractères : noms, textes, couleurs.

ex : `"Hello World"`, `"#3CBE82"`

bool

Vrai ou faux : résultat d'une comparaison, utilisé par les conditions.

ex : `True`, `False`

None

Absence de valeur : valeur par défaut, résultat « rien trouvé ».

ex : `None`

En pratique

```
count = 20                # int
ratio = 0.45              # float
title = "Hello World"    # str
is_running = True        # bool
target = None            # NoneType

print(type(ratio))        # <class 'float'>

# Conversion entre types
size = int("15")          # 15
label = str(count)        # "20"

# f-string : insère des valeurs dans un texte
print(f"Total : {count} éléments")
```

- Python est à **typage dynamique** : le type est porté par la valeur, pas par la variable.
- Nommage : **lower_snake_case** pour les variables et fonctions, **UPPER_SNAKE_CASE** pour les constantes.

1.5 Les opérateurs

Mathématiques

+ : addition
ex : `1 + 1` → `2`

- : soustraction
ex : `5 - 2` → `3`

***** : multiplication
ex : `2 * 3` → `6`

/ : division
ex : `7 / 2` → `3.5`

// : division entière
ex : `7 // 2` → `3`

% : modulo, reste
ex : `7 % 2` → `1`

****** : puissance
ex : `3 ** 2` → `9`

Comparaison

== : égalité
ex : `3 == 3` → `True`

!= : différence
ex : `3 != 3` → `False`

> : supérieur
ex : `4 > 2` → `True`

< : inférieur
ex : `4 < 2` → `False`

>= : supérieur ou égal
ex : `3 >= 3` → `True`

<= : inférieur ou égal
ex : `2 <= 3` → `True`

+= -= *= : affectation composée
ex : `x += 2` `# x = x + 2`

Logique et appartenance

and : vrai si les deux sont vrais
ex : `x > 0 and x < width`

or : vrai si l'un des deux est vrai
ex : `y == 0 or y == height - 1`

not : inverse un booléen
ex : `not is_running`

in : présence dans une collection
ex : `3 in [1, 2, 3]` → `True`

is : même objet, surtout avec None
ex : `target is None`

Exemple d'application

Une grille peut être rangée dans une simple liste, colonne par colonne. `//` et `%` permettent de passer d'une position (x, y) à un index, et inversement.

```
index = x * rows + y      # (3, 4) -> 34 si rows = 10
x = index // rows          # 34 // 10 -> 3
y = index % rows           # 34 % 10 -> 4
```

1.6 Les conditions

Exemple de condition

```
x = 10
if x > 10:
    print('X est plus grand que 10')
elif x == 10:
    print('X est égal à 10')
else:
    print('X est plus petit que 10')
```

Traduction littérale

On déclare une variable x dont la valeur est égale à 10.

Si la valeur de la variable x est plus grande que 10,
alors cette ligne est exécutée et la condition s'arrête ici.

Sinon, si la valeur de la variable x est égale à 10,
alors cette ligne est exécutée et la condition s'arrête ici.

Sinon,
cette ligne est exécutée.

Une condition est composée comme ceci

```
if condition:
    ... # exécuté si condition vaut True
elif autre_condition:
    ... # exécuté si les tests précédents valent False
else:
    ... # exécuté si tous les tests valent False
```

À retenir

- Seul le premier bloc dont la condition vaut True est exécuté.
- **elif** et **else** sont facultatifs.
- L'indentation (4 espaces) délimite les blocs : pas d'accolades en Python.
- 0, "", [] et None sont considérés comme faux dans un test.

1.6 Les boucles

La boucle for

Répète un bloc pour chaque élément d'une séquence. **range(n)** produit les entiers de 0 à n - 1.

```
columns, rows = 4, 3

# Parcourt chaque case, colonne par colonne
for x in range(columns):
    for y in range(rows):
        print(x, y)

# range(début, fin, pas)
for x in range(1, columns):    # 1, 2, 3
    print(x)
```

Parcourir avec enumerate et zip

```
colors = ["#3CBE82", "#E65A50"]
for index, color in enumerate(colors):
    print(index, color)        # 0 #3CBE82, 1 #E65A50

xs, ys = [0, 1, 2], [5, 6, 5]
for x, y in zip(xs, ys):
    print(x, y)                # 0 5, 1 6, 2 5
```

La boucle while

Répète un bloc tant qu'une condition est vraie. Utile quand le nombre de tours n'est pas connu à l'avance.

```
current = 0
end = 5

while current != end:
    current += 1
```

Contrôler une boucle

- **break** quitte immédiatement la boucle.
- **continue** passe directement au tour suivant.

```
for n in numbers:
    if n in seen:
        continue           # déjà vu : on passe au suivant
    if n == target:
        break               # cible trouvée : on s'arrête
    seen.add(n)
```

- Une boucle **while** doit finir par rendre sa condition fausse, sinon elle ne s'arrête jamais.

1.7 Les fonctions

Exemple de fonction

```
def get_index(x, y, rows):
```

```
    index = x * rows + y
```

```
    return index
```

```
result = get_index(3, 4, 10)
```

Traduction littérale

On définit une fonction qui attend trois paramètres, elle calcule l'index d'une case à partir de sa position, puis renvoie ce résultat à l'appelant.

On appelle la fonction avec des arguments : on obtient 34.

Retours multiples et valeurs par défaut

```
def get_coords(index, rows):
```

```
    return index // rows, index % rows    # renvoie un tuple
```

```
x, y = get_coords(34, 10)                # 3, 4
```

```
def scale(value, factor=2):                # valeur par défaut
```

```
    return value * factor
```

```
scale(5)                                  # 10
```

```
scale(5, factor=3)                        # 15, argument nommé
```

À retenir

- Une fonction regroupe une tâche nommée et réutilisable.
- Les paramètres sont les noms de la définition, les arguments les valeurs passées à l'appel.
- Sans **return**, une fonction renvoie **None**.
- Plusieurs valeurs renvoyées forment un tuple, que l'on déballe : **x, y = ...**
- Les variables créées dans une fonction sont locales : elles n'existent pas en dehors.
- Un bon nom commence par un verbe : **create_**, **draw_**, **get_**.

Manipuler une liste

Séquence ordonnée et modifiable. On part de `ids = [10, 20, 30]` :

```
ids[0]  premier élément, l'index commence à 0
→ renvoie 10, ids = [10, 20, 30] (inchangée)

ids[-1] dernier élément
→ renvoie 30, ids = [10, 20, 30] (inchangée)

ids[0:2] tranche, la borne de fin est exclue
→ renvoie [10, 20], ids = [10, 20, 30] (inchangée)

ids.append(40) ajoute à la fin
→ ids = [10, 20, 30, 40]

ids.insert(1, 15) insère à un index donné
→ ids = [10, 15, 20, 30, 40]

ids.remove(15) retire la première occurrence de 15
→ ids = [10, 20, 30, 40]

ids.pop() retire et renvoie le dernier élément
→ renvoie 40, ids = [10, 20, 30]

ids.pop(0) retire et renvoie l'élément à l'index 0
→ renvoie 10, ids = [20, 30]

len(ids) nombre d'éléments
→ renvoie 2, ids = [20, 30] (inchangée)

20 in ids teste la présence
→ renvoie True, ids = [20, 30] (inchangée)
```

Compréhension de liste

```
# Boucle classique
positions = []
for x in range(3):
    positions.append((x, 0))

# Même résultat avec une compréhension de liste
positions = [(x, 0) for x in range(3)]

# Avec un filtre
even = [n for n in range(10) if n % 2 == 0]
```

Référence ou copie ?

```
a = [1, 2, 3]
b = a                # même liste, deux noms
b.append(4)
print(a)             # [1, 2, 3, 4]

c = a.copy()         # copie indépendante
a[:] = [0, 0]        # remplace le contenu sur place
```

1.8 Tuples et dictionnaires

Le tuple

Séquence ordonnée *non modifiable*, pour des valeurs qui vont ensemble : coordonnées, dimensions, déplacement.

```
SCREEN_SIZE = (1280, 720)
width, height = SCREEN_SIZE      # déballage

move = (1, -1)                   # dx, dy
dx, dy = move

SCREEN_SIZE[0] = 1920             # TypeError : non modifiable
```

- Un tuple peut servir de *clé de dictionnaire* ou d'élément de set, contrairement à une liste.
- `a, b = b, a` échange deux variables grâce au déballage.

Le dictionnaire

Associe des *clés* uniques à des *valeurs*. On accède directement à une valeur par sa clé, sans parcourir.

```
colors = {
    "red": "#E65A50",
    "green": "#3CBE82",
}

colors["red"]                # "#E65A50"
colors["yellow"] = "#F5B83D" # ajoute une entrée
colors.get("blue", "#303844") # défaut si absente
"green" in colors            # True

for name, color in colors.items():
    print(name, color)
```

- Une clé absente lue avec `[]` lève une **KeyError**, `get()` renvoie une valeur par défaut.
- `keys()`, `values()` et `items()` permettent de parcourir le dictionnaire.

1.8 Sets et comparatif

Le set

Ensemble de valeurs **uniques**, sans ordre. Le test d'appartenance **in** y est très rapide, même sur de gros volumes.

```
seen = {0, 1, 2}
seen.add(3)
seen.add(3)                # pas de doublon
5 in seen                  # False

a = {1, 2, 3}
b = {2, 3, 4}
a & b                      # {2, 3} intersection
a | b                      # {1, 2, 3, 4} union
a - b                      # {1} différence
```

- **{}** crée un dict vide : pour un set vide, écrire **set()**.
- Usage courant : mémoriser les éléments déjà traités pour ne jamais y repasser.

Quelle collection choisir ?

list []

Ordonnée, modifiable, doublons autorisés.

Usage : Une suite d'éléments à parcourir.

tuple ()

Ordonnée, non modifiable.

Usage : Une position (x, y), des dimensions.

dict { : }

Clés uniques vers valeurs, modifiable.

Usage : Une valeur associée à chaque nom.

set { }

Sans ordre, valeurs uniques, modifiable.

Usage : Les éléments déjà traités.

Se demander : faut-il un **ordre**, pouvoir **modifier**, garantir l'**unicité** ou accéder par une **clé** ?

1.9 Classes et instances

Exemple de classe

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

a = Point(0, 5)

print(a.x)
```

Traduction littérale

On définit la **classe** Point : le plan de construction d'un point.

`__init__` s'exécute à la création et reçoit l'objet (**`self`**).

On range x dans un **attribut** de l'objet,
ainsi que y.

On crée une **instance** : un point concret en (0, 5).

On lit un attribut de l'instance : affiche 0.

Vocabulaire

- Classe : le modèle, ici **Point**.
- Instance : un objet créé à partir de la classe, ici **a**.
- Attribut : une donnée portée par l'objet, **a.x**.
- Méthode : une fonction définie dans la classe, appelée sur l'objet.
- **self** : l'instance courante, premier paramètre de chaque méthode.

Ajouter une méthode

```
import math

class Point:
    ...
    def distance_to(self, other):
        return math.hypot(other.x - self.x, other.y - self.y)
```

Appel : **a.distance_to(b)**. Python passe automatiquement **a** comme **self**.

1.9 Les énumérations

Définir une Enum

Une **Enum** définit un ensemble **fermé** de valeurs nommées : un état ou un type qui ne peut prendre que quelques valeurs connues.

```
from enum import Enum

class Direction(Enum):
    UP = 0
    DOWN = 1
    LEFT = 2
    RIGHT = 3

move = Direction.UP
if move == Direction.UP:
    print(move.name)      # "UP"
    print(move.value)     # 0

for d in Direction:      # parcourt toutes les valeurs
    print(d.name)
```

Pourquoi pas une simple chaîne ?

```
# Avec une chaîne : la faute passe inaperçue
move = "up"
if move == "Up":          # jamais vrai, aucune erreur
    ...

# Avec une Enum : erreur immédiate
move = Direction.Up       # AttributeError
```

- Les valeurs possibles sont listées en un seul endroit.
- Une faute de frappe provoque une **erreur immédiate** au lieu d'un bug silencieux.
- **.name** donne le nom du membre, **.value** la valeur associée.

1.10 Modules et imports

Organisation d'un projet

```
mon_projet/
├── .venv/           # environnement local
├── main.py          # point d'entrée
├── shapes.py        # classe Point
├── requirements.txt # dépendances
└── .gitignore       # fichiers ignorés par Git
```

- Chaque fichier **.py** est un module.
- Un module, une responsabilité : on sait où chercher et quoi modifier.

Point d'entrée

```
def main():
    print("Démarrage")

if __name__ == "__main__":
    main()
```

Ce bloc ne s'exécute que si le fichier est lancé directement (**python main.py**), pas s'il est importé.

Les formes d'import

```
# Module de la bibliothèque standard
import random
random.choice([-1, 0, 1])

# Bibliothèque externe, avec un alias plus court
import matplotlib.pyplot as plt
plt.show()

# Noms précis depuis un module du projet
from shapes import Point
p = Point(0, 5)
```

- Bibliothèque standard : fournie avec Python (**random, math, enum**).
- Bibliothèque externe : installée avec pip dans le **.venv** (**matplotlib**).
- Module du projet : un fichier **.py** du même dossier.
- Éviter **from module import *** : on ne sait plus d'où vient chaque nom.

1.11 Matplotlib

Vérification

Matplotlib a été installé dans le `.venv` à l'étape 4 de l'installation, et vérifié à l'étape 5 : il est prêt à être importé.

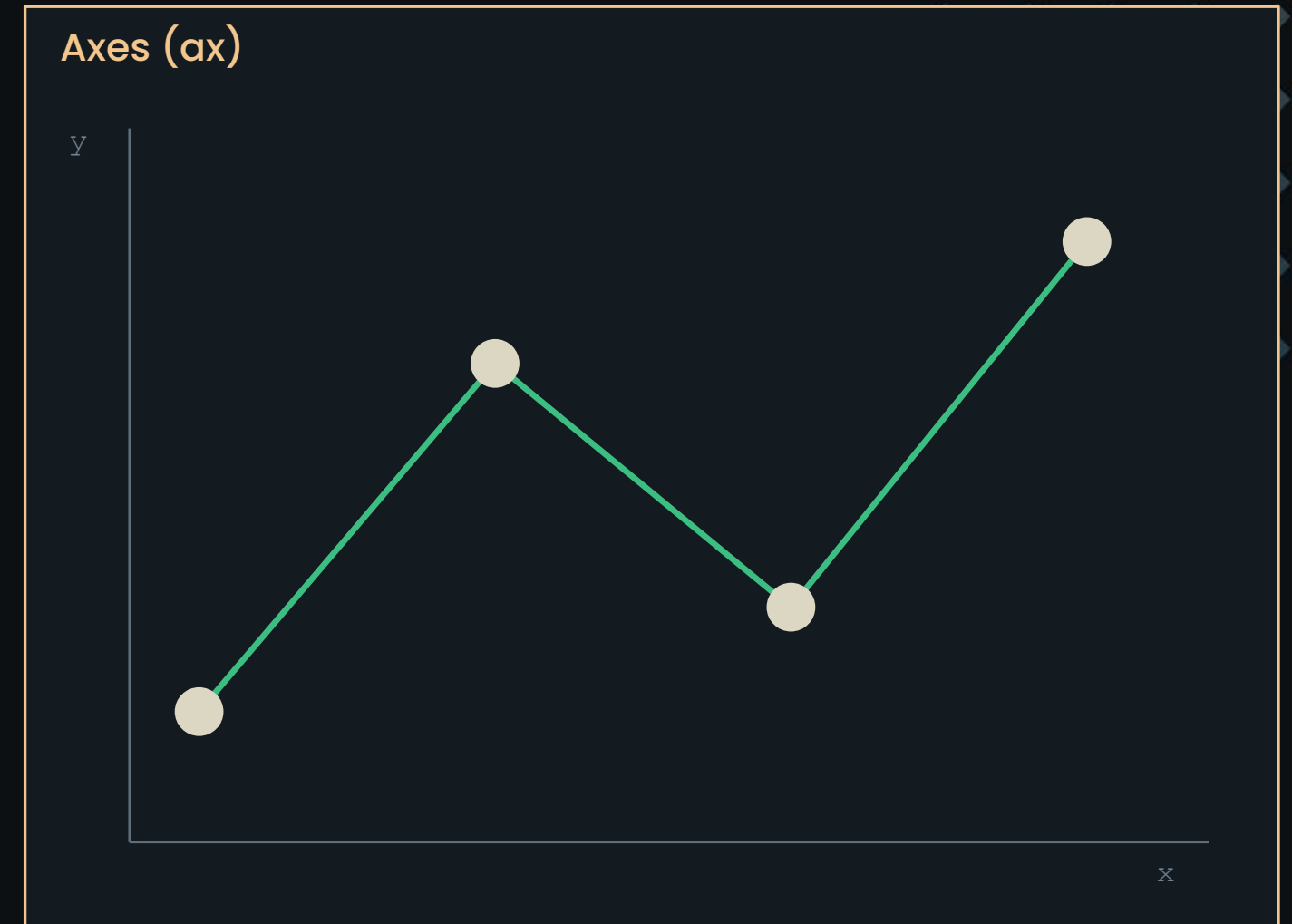
Deux objets à connaître

- Figure (**fig**) : la fenêtre entière, qui contient tout le reste.
- Axes (**ax**) : une zone de dessin avec son repère. On y dessine points et segments.
- `plt.subplots()` crée les deux, `plt.show()` ouvre la fenêtre et attend sa fermeture.

Dessiner

```
ax.plot(x, y, marker="o")      # un point  
ax.plot([x1, x2], [y1, y2])   # un segment
```

Figure (fig)



La figure contient un ou plusieurs axes. Les coordonnées (x, y) sont celles de l'axe, pas des pixels.

1.11 Premier script

test_matplotlib.py

```
import matplotlib.pyplot as plt

# Taille de la grille utilisée pour le test
columns, rows = 5, 3

fig, ax = plt.subplots()
fig.canvas.manager.set_window_title("Test Matplotlib")

# Dessine une petite grille de points
for x in range(columns):
    for y in range(rows):
        ax.plot(x, y, marker="o", color="#303844")

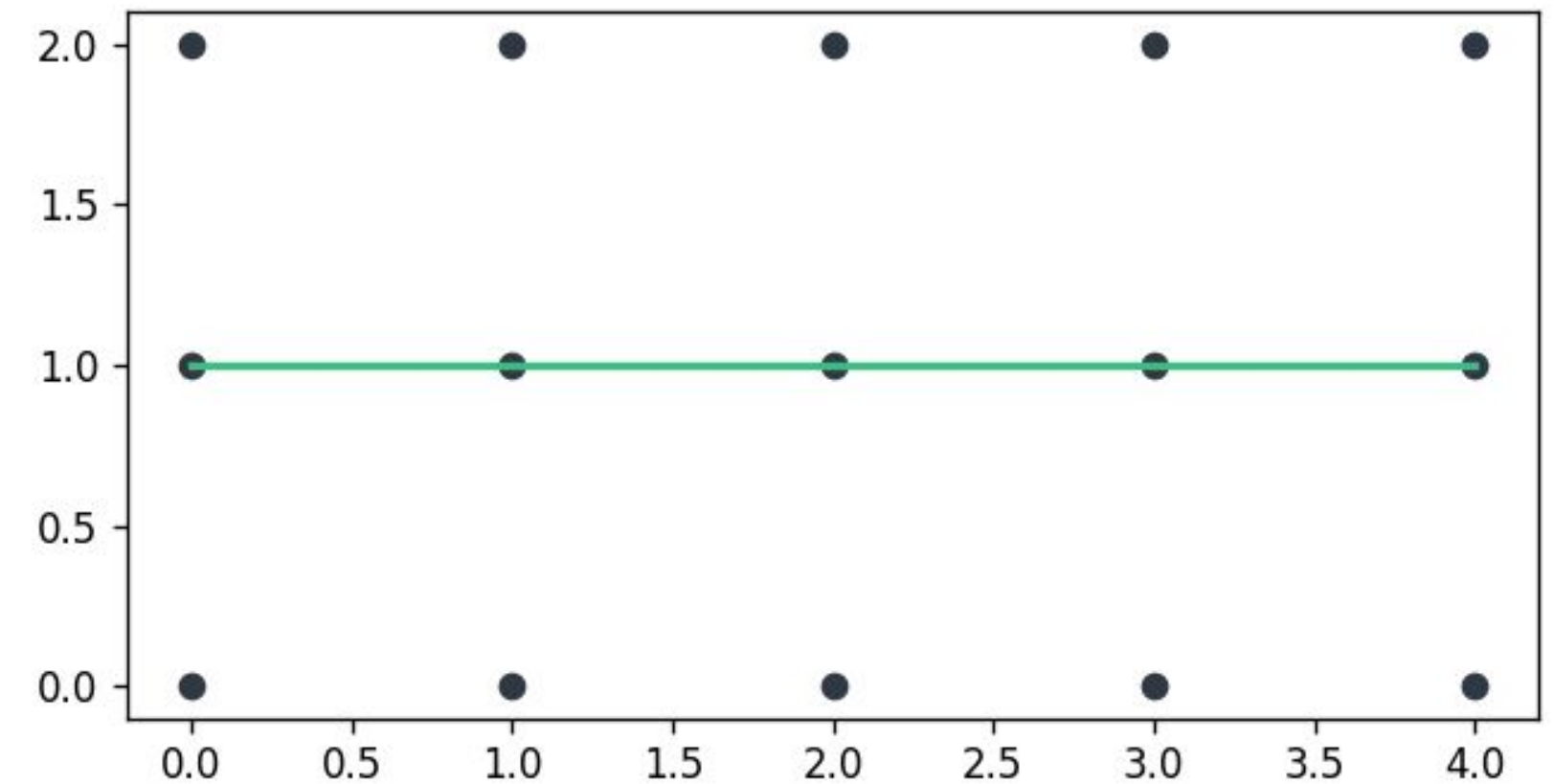
# Dessine un segment entre deux points
ax.plot([0, 4], [1, 1], color="#3CBE82", linewidth=2)

ax.set_aspect("equal")
plt.show()
```

- Lancer le script avec le bouton ▶ de VS Code (interpréteur du `.venv`), ou dans le terminal avec le Python du `.venv`.
- Une fenêtre s'ouvre : l'environnement est validé.

Résultat attendu

Test Matplotlib



- Une grille de 5×3 points et un segment vert.
- Le titre de la fenêtre correspond à `set_window_title`.

1.12 Recap

Checklist de validation

- ☐ `python --version` affiche Python 3.14.
- ☐ Le dossier du projet contient un `.venv` créé localement.
- ☐ `requirements.txt` liste Matplotlib, installé dans le `.venv`.
- ☐ VS Code utilise l'interpréteur du `.venv`.
- ☐ Le premier script ouvre une fenêtre Matplotlib.
- ☐ Les groupes de 3 étudiants sont constitués.

Livrables

- Environnement Python fonctionnel avec VS Code et `.venv`.
- Matplotlib installé et fenêtre de test fonctionnelle.

Notions revues

- Variables, types et opérateurs.
- Conditions, boucles et fonctions.
- Collections : list, tuple, dict et set.
- Classes, instances, Enum, modules et imports.
- Figure et axes Matplotlib.

2. Architecture & génération procédurale

Objectifs

- Comprendre comment représenter un réseau routier sous forme de graphe.
- Structurer les données avec des classes Python.
- Créer des fonctions pour construire et visualiser le graphe.
- Mettre en place une première génération procédurale du réseau.

Livrables

- Visualisation Matplotlib d'une grille de nodes.
- Premier chemin routier procédural fonctionnel.
- Base de projet commune prête à être développée en autonomie.

Durée

- 3h30

Déroulé

- Introduction au projet RoadNetwork et réflexion sur la représentation d'un réseau routier.
- Découverte de Matplotlib : figure, axes, coordonnées, points, segments et affichage.
- Création des classes Node et Segment pour organiser les données.
- Création des types de Node : Unused, Start, End, Connection et Intersection.
- Création des fonctions de construction et d'affichage : `create_node()`, `create_segment()`, `draw_node()`.
- Génération d'une grille de nodes Unused à partir de rows et columns.
- Définition du Start au milieu de la première colonne.
- Génération d'un chemin principal colonne par colonne avec déplacements contraints.
- Ajout d'un bouton Randomize dans la fenêtre.
- Réflexion collective sur les modifications permettant de complexifier le réseau en autonomie.

3. Features & visualisation

Objectifs

- Faire évoluer le réseau à partir du socle commun.
- Tester et corriger un système de génération procédurale.
- Consolider l'autonomie en conception et en développement Python.
- Ajouter des fonctionnalités avancées selon l'avancement.

Livrables

- Réseau procédural finalisé et fonctionnel.
- Projet testé et sans erreur bloquante.
- Features supplémentaires intégrées selon l'avancement du groupe.

Durée

- 3h30

Déroulé

- Reprise du projet et finalisation des modifications conçues en fin de séance précédente.
- Développement autonome : branches, intersections, routes secondaires ou autres évolutions du réseau.
- Tests répétés de la génération afin d'identifier les cas limites et incohérences.
- Correction des erreurs : sorties de grille, connexions invalides, croisements ou générations impossibles.
- Utilisation d'une seed pour reproduire et diagnostiquer un résultat procédural si nécessaire.
- Amélioration de la lisibilité et de l'organisation du code.
- Ajout de features avancées selon l'avancement : contraintes supplémentaires, visualisation enrichie, recherche de chemin, trafic ou autres propositions pertinentes.
- Préparation d'une version stable du projet pour la restitution.

4. Préparation & soutenance

Objectifs

- Structurer une présentation technique courte.
- Être capable d'expliquer l'architecture et l'algorithme réalisés.
- Présenter les évolutions apportées au socle commun.
- Défendre ses choix techniques et remettre le projet.

Livrables

- Projet RoadNetwork final.
- Support de présentation (exporté).
- Présentation orale et démonstration du projet.

Durée

- 3h30

Déroulé

- Finalisation et vérification du projet avant remise.
- Préparation d'un support de présentation synthétique : problème, architecture, algorithme, features, difficultés et résultat.
- Préparation d'une démonstration du générateur et des fonctionnalités développées.
- Anticipation des questions techniques sur le code et les choix de conception.
- Passage à l'oral : présentation, démonstration, questions/réponses et feedback.
- Remise de l'archive du projet et du support de présentation.

Architecture & code Organisation des données et classes / 3 Fonctions et structure du programme / 2 Lisibilité et qualité du code / 2
Génération procédurale Réseau fonctionnel / 2 Cohérence des règles de génération / 2 Évolutions apportées au socle / 2
Visualisation Représentation visuelle / 3 Animation / 1
Soutenance Présentation et démonstration / 1 Compréhension technique / 2

/ 7

/ 6

/ 4

/ 3

Rendu

- Une archive ZIP du projet Python
- Le support de présentation (exporté)

/ 20

Bonus Features avancées : 0.5 Algorithme avancé : 0.5 Visualisation enrichie : 0.5 Documentation impeccable : 0.5
--

/ 3

Malus Retard de rendu : -3 Erreur bloquante au lancement : -3 Réutilisation inappropriée de code brut : -2
--

/ -8